

MLB Statcast 데이터를 활용한 LSTM 기반 구종·위치 추천 시스템



지능기전공학부 무인이동체공학전공 20xxxxxx 지준구

목차

01	프로젝트 개요	08	실험 설정 및 평가 지표
02	문제 정의 및 최종 목표	09	실험 결과
03	최종 시스템 구성도	10	모델 비교 분석
04	데이터셋	11	웹 데모 시스템
05	데이터 전처리·시퀀스 구성	12	한계 및 후속 연구
06	모델 및 모듈 설계	13	결론
07	추천 구조	14	참고 문헌

프로젝트 개요

／
야구의 볼배합은 단순히 다음 구종 하나를 고르는 문제가 아니라, **이전 투구 흐름과 현재 경기 상황을 함께 고려하는 순차적 의사결정 문제**이다.

같은 슬라이더라도 0B-2S 유인구와 3B-1S 승부구는 전략적 의미가 달라질 수 있다.

기존의 경험·직관 중심 판단을 보완하기 위해 **투구 단위 데이터를 일관되게 활용할** 필요가 있다.

／
MLB Statcast pitch-level 데이터를 활용해 이전 5개 투구의 구종·zone과 현재 경기 상황을 입력으로 구성하였다.

LSTM과 Transformer Encoder를 비교하고, 최종적으로 **LSTM Top-3 구종 후보를 위치 추천 모듈**에 연결하였다.

최종 산출물은 구종과 Statcast zone을 함께 제안하는 **웹 기반 추천 데모**이다.

문제 정의 및 프로젝트 목표

문제 정의

입력 맥락 구성

같은 타석 내 이전 5개 투구의 구종과 Statcast zone, 볼카운트·아웃·이닝·주자상황을 함께 사용

다음 구종 예측

현재 투구의 pitch_type을 예측 대상으로 두고 미래 정보를 입력에 포함하지 않음

위치 추천 확장

예측된 구종 후보에 대해 delta_run_exp 기반으로 투수에게 유리했던 zone 후보를 정렬

최종 구현 목표

데이터 파이프라인

pybaseball로 약 228만 개 투구 데이터를 수집하고 전처리·시퀀스 데이터를 생성

모델 비교 실험

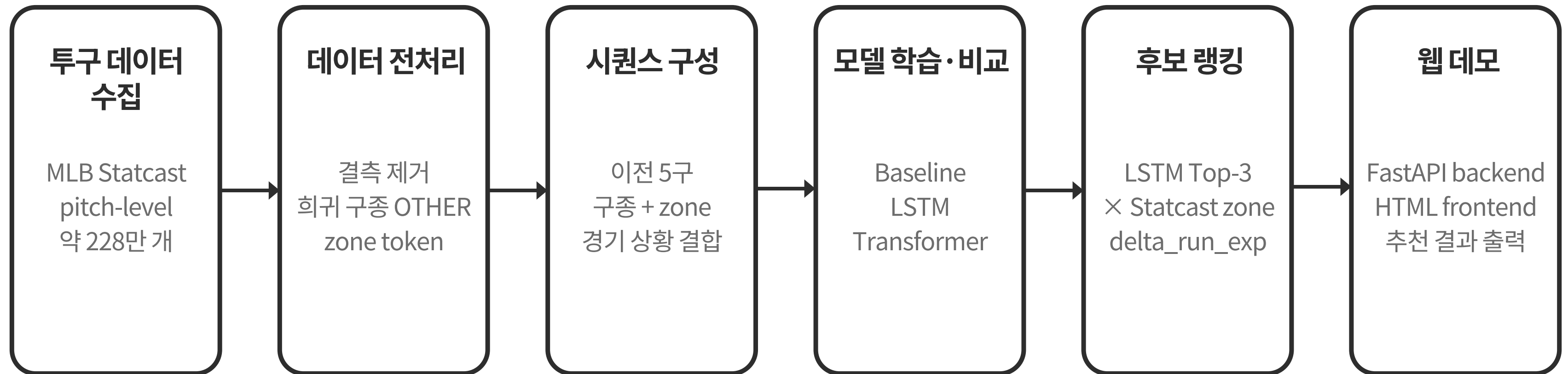
6개 baseline, LSTM, Transformer Encoder를 Accuracy·Top-3·Macro F1 기준으로 비교

웹 데모 구현

FastAPI 백엔드와 HTML 프론트엔드를 통해 입력→추천 결과 출력을 연결

추천 결과는 완전한 최적 볼배합이 아니라, 공개 Statcast 데이터 기반 의사결정 보조 결과로 해석한다.

최종 시스템 구성도



투구별 데이터를 시퀀스 형태로 재구성한 뒤, 구종 후보 생성과 위치 후보 랭킹을 연결하여 실제 추천 화면까지 구현하였다.

데이터셋

항목	내용
데이터 출처	MLB Baseball Savant Statcast Pitch-by-Pitch 데이터
수집 도구	Python pybaseball 라이브러리의 statcast 함수
데이터 단위	투구 1개 단위의 pitch-level 데이터
수집 기간	2023.03.01 ~ 2025.09.27, 정규시즌 경기
데이터 규모	약 228만 개 투구 데이터
주요 변수	pitch_type, zone, balls/strikes, outs, inning, runners, pitcher, stand/p_throws, delta_run_exp
활용 목적	이전 투구 흐름과 현재 경기 상황을 반영한 시퀀스 데이터 구성

투구 하나를 하나의 관측치로 보고, 이전 투구 흐름과 현재 경기 상황을 모두 모델 입력으로 활용하였다.

데이터 전처리·시퀀스 구성

-  pitch_type 결측 행 제거
-  game_date·game_pk·at_bat_number·pitch_number 기준 정렬
-  on_1b/on_2b/on_3b → runner_1b/2b/3b 이진 변수
-  희귀 구종과 특수 구종을 OTHER로 통합
-  Statcast zone을 Z1~Z14 token으로 변환
-  vocabulary·scaler는 train 데이터 기준으로 생성

데이터 전처리·시퀀스 구성

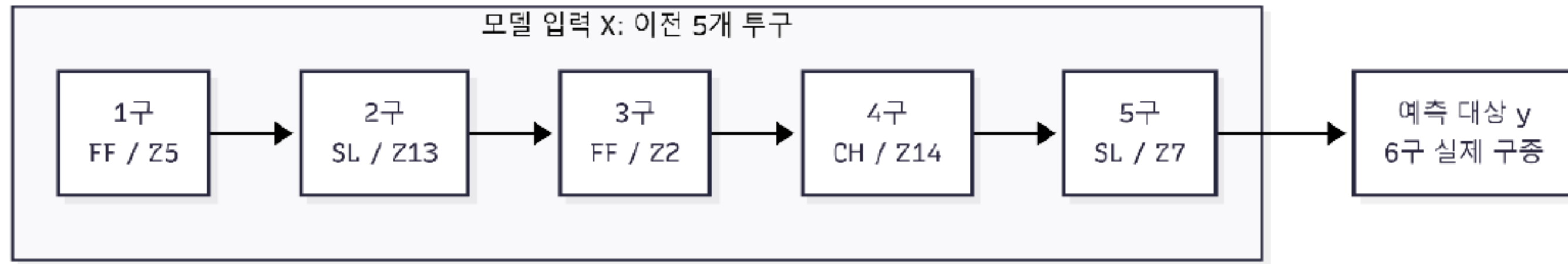


그림 3-1. 시퀀스 데이터 구성 예시

시퀀스 구성 방식

- 같은 타석 내 현재 투구를 예측하기 위해, 현재 시점 이전의 최대 5개 투구만 사용
- 구종 시퀀스: PAD
- zone 시퀀스: ZPAD
- 길이가 부족하면 왼쪽 padding으로 고정 길이 구성

전처리된 pitch-level 데이터는 이전 5구의 순서와 현재 경기 상황이 결합된 공통 입력 데이터로 변환되었다.

모델 및 모듈 설계



Baseline

Global Majority
Count/Base Majority
Pitcher Majority
Pitcher + Count
Previous Pitch Markov
Previous Pitch + Count

딥러닝 모델의 비교 기준



LSTM

구종 · zone embedding 결합,
이전 5구의 순차 흐름을 요약

마지막 hidden state
+ 경기 상황
+ 투수/좌우 정보 embedding

최종 시스템의 구종 후보
생성기



Transformer Encoder

구종 · zone embedding
+
positional embedding

Self-attention으로
시퀀스 내 관계 학습

LSTM과 동일 입력
조건에서 비교



delta_run_exp Ranker

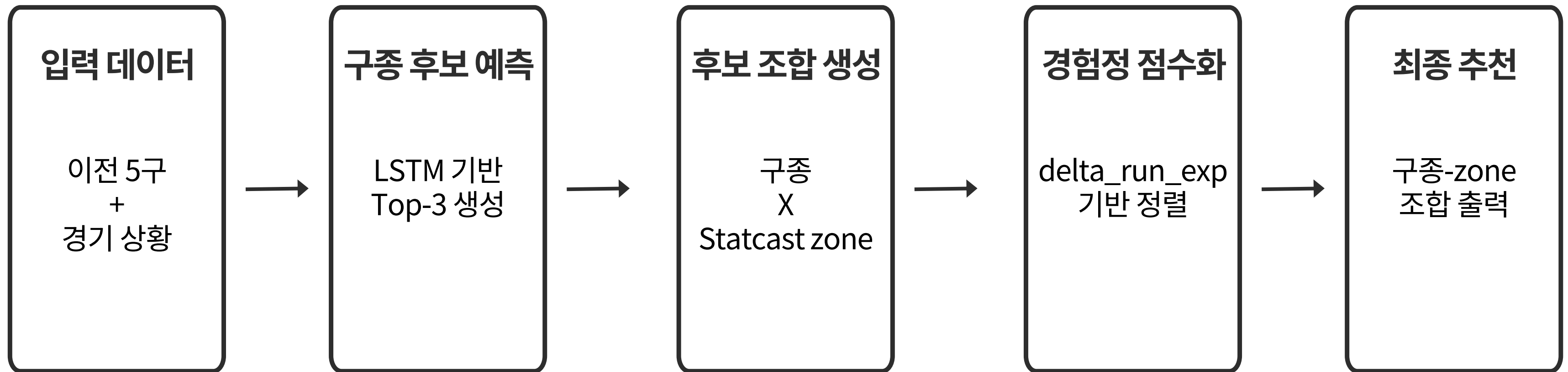
LSTM Top-3 구종 후보와
가능한 Statcast zone을
조합

과거 delta_run_exp
평균을 기준으로
구종-zone 후보를 정렬

표본 부족 시 단계적
fallback

분류 모델은 다음 구종 후보를 만들고, 경험적 랭킹 모듈은 후보별 위치를 정렬하는 역할을 담당한다.

추천 구조



구종 후보 생성: LSTM이 이전 5개 구종·zone과 현재 경기 상황을 바탕으로 다음 구종 확률을 산출하고, 상위 3개 후보를 선택한다.

위치 후보 정렬: 각 구종 후보와 가능한 Statcast zone을 조합한 뒤, 과거 train 데이터의 delta_run_exp 평균을 점수로 사용한다.

최종 출력: 점수가 낮을수록 투수 관점에서 유리했던 조합으로 해석하여 구종-zone 추천 순위를 제공한다.

정밀한 인과 최적화가 아니라, 과거 결과 경향을 바탕으로 후보를 정렬하는 하이브리드 추천 구조이다.

성능 평가 방안

모델 구분	분할 방식	사용 목적
Baseline	Train 80% / Test 20%	조건별 최빈 구종 lookup 생성 및 평가
LSTM	Train 70% / Valid 10% / Test 20%	학습, 모델 선택, 최종 평가
Transformer Encoder	Train 70% / Valid 10% / Test 20%	LSTM과 동일 조건 비교
delta_run_Exp 랭킹	Train 80% / Test 20%	경험적 통계 생성 및 outcome 평가

항목	설정
입력 길이	이전 5개 투구
추가 입력	balls, strikes, outs, inning, runners, pitcher, stand, p_throws
학습 설정	Epoch 8 / Batch 512 / LR 0.001 / AdamW
불균형 보정	Class weight 적용
모델 선택	Validation Macro F1 기준

-  **Accuracy**: Top-1 예측이 실제 구종과 일치한 비율
-  **Top-3 Accuracy**: 상위 3개 추천 후보 안에 실제 구종이 포함되는 비율
-  **Macro F1**: 구종 class별 성능을 동일 비중으로 평균하여 클래스 불균형을 고려
-  **MAE / RMSE / R²**: delta_run_exp 기반 zone 랭킹 모듈의 outcome 점수 오차 확인

단일 정답 정확도뿐 아니라 추천 후보 생성 능력과 소수 구종에 대한 균형 성능을 함께 평가하였다.

실험 결과

모델	Accuracy	Top-3 Accuracy	Macro F1	해석
Pitcher + Count Baseline	0.4060	0.8005	0.3329	Top-1 기준 가장 강한 baseline
LSTM	0.3896	0.8353	0.3972	Top-3 Accuracy와 Macro F1 기준 최종 선택
Transformer Encoder	0.3722	0.8293	0.3896	LSTM과 동일 조건 비교

zone 랭킹 지표	값	해석
MAE	0.1314	평균 절대 오차
RMSE	0.2408	큰 오차까지 고려
R ²	0.0088	정밀 회귀보다는 후보 랭킹 용도

LSTM은 단일 예측 정확도보다 추천 후보 생성 능력과 class 균형 성능에서 강점을 보였다.

결과 해석

- 👉 Pitcher + Count Majority는 Top-1 Accuracy 기준 가장 강함
- 👉 투수별 성향과 볼카운트가 구종 선택에 매우 중요한 변수임을 확인
- 👉 LSTM은 Accuracy는 낮지만 Top-3 Accuracy와 Macro F1에서 우수
- 👉 추천 시스템 관점에서는 실제 구종이 후보군 안에 포함되는 능력이 중요
- 👉 Transformer는 입력 길이 5의 짧은 시퀀스에서 LSTM보다 낮은 성능

LSTM을 구종 후보 생성기로 선택하였으며,
최종 구조는 LSTM Top-3 구종 후보 + delta_run_exp 기반 zone 랭킹 이다.

본 프로젝트의 목표는 실제 선택 구종 하나를 무조건 맞히는 것이 아니라, 가능성 있는 구종·위치 후보를 제안하는 것이다.

웹 데모 시스템 구현

① 경기 상황 입력

구종·위치 추천 시스템

이전 투구 흐름과 현재 경기 상황을 바탕으로 다음 구종 후보를 예측하고, Statcast zone별 기대득점 변화 경향을 이용해 투수에게 유리한 구종·위치 조합을 추천합니다.

LSTM 기반 구종 후보 생성

기대득점 변화 기반 위치 선정

투수 시점 Zone UI

직접 선택 비교

현재 경기 상황

현재 타석의 상황과 투수·타자 좌우 정보를 입력합니다.

데모 투수 기준

현재 화면은 학습 데이터에 포함된 예시 투수를 기준으로 추천합니다. 실제 서비스에서는 팀/투수 선택 기능으로 확장할 수 있습니다.

볼

1

스트라이크

2

아웃카운트

1

이닝

3

1루 주자

없음

2루 주자

있음

3루 주자

없음

타자 방향

우타자

투수 손

우투수

최종 추천 개수

5

후보 구종 제한

☒ 포심 패스트볼 (4-Seam Fastball)

☐ 싱커 (Sinker)

☒ 슬라이더 (Slider)

☐ 체인지업 (Changeup)

☐ 커터 (Cutter)

☐ 스위퍼 (Sweeper)

☒ 커브 (Curveball)

☐ 스플리터 (Splitter)

☐ 너클커브 (Knuckle Curve)

선택한 구종만 최종 추천 후보로 평가합니다. 선택하지 않으면 LSTM Top-3와 해당 투수의 주 구종을 자동 후보로 사용합니다.

현재 모델 범위

현재 데모는 이전 투구 흐름, 볼카운트, 주자 상황, 투수 손, 타자 방향을 중심으로 추천합니다. 점수, 자, 오늘 투구 수, 선발/구원 여부, 구속, 회전수는 아직 직접 반영하지 않았으며 향후 확장 항목입니다.

추천 보기

예시값 초기화

② 이전 투구 zone 선택

2. 이전 투구 이력과 위치

구종을 선택하고, 투수 시점의 스트라이크존에서 위치를 골라줍니다.

선택 대상: 9구 전 위치

투수 시점 표시

타자

타자

투수

투수

Zone 안내

Z1~Z9는 스트라이크존 내부, Z11~Z14는 존 밖 영역입니다. 화면은 중계에서 획득한 투수 시점의 가깝게 좌우를 배치했습니다.

이전 투구 입력

5구 전

포심 패스트볼 (4-Seam Fastball)

Z5

위치
선택

4구 전

슬라이더 (Slider)

Z7

위치
선택

3구 전

커브 (Curveball)

Z1

위치
선택

2구 전

슬라이더 (Slider)

Z12

위치
선택

1구 전

포심 패스트볼 (4-Seam Fastball)

Z11

위치
선택

1~4구전 상황에서는 부족한 이전 투구를 이전 투구 알음·모름 설정할 수 있습니다. 서버에는 실제 존재하는 이전 투구만 전달되고, 부족한 부분은 해당 내부에서 PAD로 처리됩니다.

③ 추천 결과 출력

3. 추천 결과

LSTM 구종 후보와 기대득점 변화 기반 최종 구종·위치 추천을 확인합니다.

LSTM Top-3 구종 후보

순위	구종	예측 확률
1	체인지업 Changeup	26.69%
2	커브 Curveball	23.54%
3	슬라이더 Slider	22.53%

최종 구종·위치 추천

순위	구종	위치	예상 기대득점 변화	근거	표본
1	포심 패스트볼 4-Seam Fastball	Z9	-0.03011 투수에게 유리	같은 카운트구종·위치 기준	1,054 신뢰도 높음
2	포심 패스트볼 4-Seam Fastball	Z7	-0.02911 투수에게 유리	같은 카운트구종·위치 기준	913 신뢰도 보통
3	슬라이더 Slider	Z9	-0.02362 투수에게 유리	같은 카운트구종·위치 기준	2,147 신뢰도 높음
4	슬라이더 Slider	Z3	-0.02003 투수에게 유리	같은 카운트구종·위치 기준	515 신뢰도 보통
5	슬라이더 Slider	Z1	-0.01829 투수에게 유리	같은 카운트구종·위치 기준	432 신뢰도 보통

신뢰도 기준

추천 근거는 학습 구간 약 132만 개 투구에서 계산한 경험적 기대득점 변화입니다. 표본 수 1,000개 이상은 높음, 100~999개는 보통, 100개 미만은 낮음으로 표시합니다.

4. 직접 선택 비교

비교할 구종

포심 패스트볼 (4-Seam Fastball)

비교할 위치

Z13

비교 위치 선택

직접 선택 비교

Raw JSON 보기

FastAPI 백엔드가 LSTM 모델과 ranker artifact를 로드하고, HTML 프론트엔드에서 입력·추천·직접 비교 기능을 제공한다.

한계 및 후속 연구

현재 한계

- ① pitch_type은 실제 선택 구종이며 항상 최적 선택은 아님
- ② zone은 포수의 의도 위치가 아니라 실제 투구 도착 위치
- ③ FF·SL·SI 등 다수 구종 중심의 class 불균형 존재
- ④ delta_run_exp는 타자·수비·타구질·경기 흐름의 영향을 함께 받음
- ⑤ 점수차, 당일 투구수, 구속, 회전수 등 세부 feature 미반영

후속 연구 방향

- ① 점수차·투구수·구속·회전수·무브먼트 등 투구 품질 feature 추가
- ② 타자별 구종 대응 능력과 zone별 약점 반영
- ③ 구종과 위치를 동시에 예측하는 multi-task learning 확장
- ④ 타석 전체 결과를 고려하는 전략 평가 또는 강화학습 적용
- ⑤ 투수·타자 선택, 실시간 경기 상황 입력, 추천 신뢰도 시각화 확장

현재 시스템은 공개 데이터 기반의 추천 보조 도구이며, 실제 경기 적용에는 현장 맥락과 추가 검증이 필요하다.

결론 및 활용 가능성

최종 성과

- ① 약 228만 개 MLB Statcast pitch-level 데이터 수집
- ② 결측 제거, 희귀구종 OTHER 통합, zone token 변환
- ③ 같은 타석 내 이전 5개 구종·zone 시퀀스 구성
- ④ 6개 baseline, LSTM, Transformer Encoder 비교
- ⑤ LSTM Top-3 + delta_run_exp zone 랭킹 결합
- ⑥ FastAPI 백엔드와 HTML 프론트엔드 웹 데모 구현

활용 가능성

경기 전략 분석 보조

포수 리드 및 투수 운영 참고 자료

투수별 구종 패턴 분석

야구 데이터 교육용 시각화 데모

향후 실시간 경기 분석 시스템으로 확장

단일 정답 예측을 넘어, 구종 후보와 위치 후보를 함께 제안하는 딥러닝 응용 프로젝트로 활용.

참고 문헌

- [1] MLB Baseball Savant. Statcast Search. https://baseballsavant.mlb.com/statcast_search
- [2] MLB.com. Statcast Glossary. <https://www.mlb.com/glossary/statcast>
- [3] pybaseball Developers. pybaseball: Baseball data scraping and analysis tools in Python. <https://github.com/jldbc/pybaseball>
- [4] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- [5] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*.
- [6] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*.
- [7] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [8] FastAPI Developers. FastAPI Documentation. <https://fastapi.tiangolo.com/>
- [9] Jun9u. pitch_project GitHub Repository. https://github.com/Jun9u/pitch_project

감사합니다.